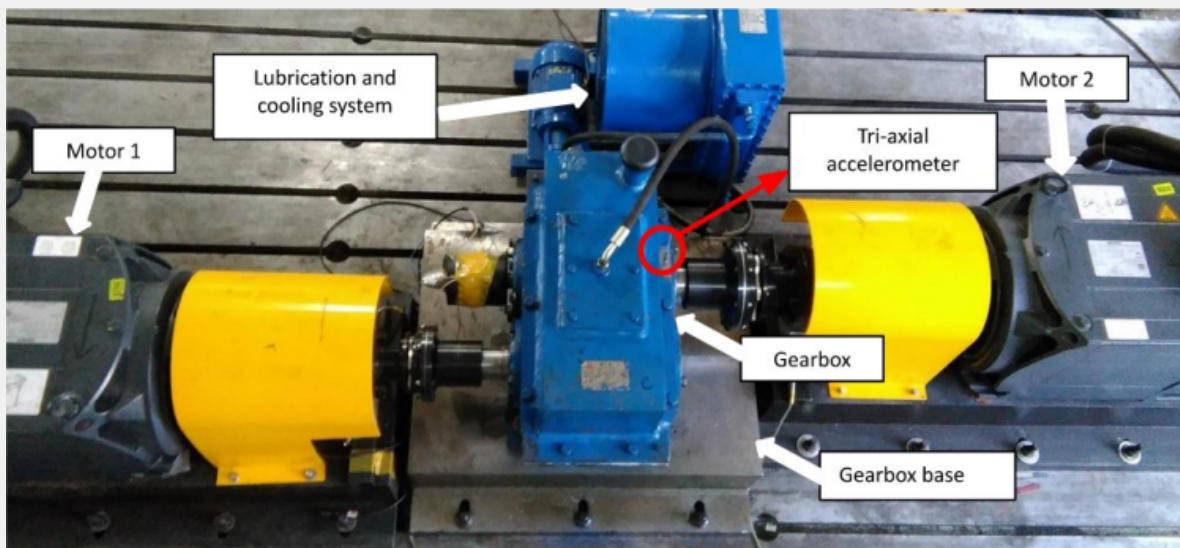


PHM AMERICA 2023

Manutenzione preventiva per la robotica e l'automazione intelligente



Autori:

Gabriel Piercecchi

Tosca Pierro

Anno accademico 2023-2024

Indice

1	Introduzione	1
1.1	Descrizione del Dataset	1
1.2	Condizioni Operative e Denominazione dei File	1
1.3	Caratteristiche del Segnale di Vibrazione	2
1.4	Durata dell'Acquisizione e Segmentazione	2
1.5	Dati di Degradazione e Livelli di Salute	3
1.6	Banco di Prova e Strumentazione	3
1.7	Obiettivi dell'Analisi	4
1.8	Procedure utilizzate	4
1.9	Repository	6
2	Addestramento CNN-AUTOENCODER	7
2.1	Introduzione	7
2.2	Procedimento	7
2.2.1	STEP 1 - Caricamento e Pre-elaborazione dei Dati	7
2.2.2	STEP 2 - Addestramento dell'Autoencoder CNN	7
2.2.3	STEP 3 - Caricamento dei dati unhealthy	9
2.2.4	STEP 4 - Valutazione degli Errori e Mapping in Livello	10
2.2.5	STEP 5 - Analisi dei Meta-dati Operativi	11
2.2.6	STEP 6 - Generazione dei File di Submission per i Dati Unhealthy	13

2.2.7 STEP 7 - Caricamento dei dati di test e generazione del submission_test	13
2.2.8 STEP 8 - Tuning e Analisi dei Threshold	15
2.3 Osservazioni Finali	15
3 Addestramento CNN con SVM	17
3.1 Introduzione	17
3.2 Parametri e Configurazione	17
3.3 Caricamento e Segmentazione dei Dati	18
3.3.1 Preprocessing e Normalizzazione	18
3.3.2 Suddivisione Training/Validation e Definizione del Modello CNN	18
3.3.3 Visualizzazione e Valutazione del Modello	19
3.3.4 Predizione sul Test Dataset	19
3.4 Modulo di Anomaly Detection Esteso	20
3.4.1 Calcolo classe predetta	21
3.5 Esportazione dei Risultati	21
4 Risultati	22
4.1 Addestramento	22
4.2 Confusion matrix	23
4.3 Prestazioni del modello	24
4.3.1 Metriche di Classificazione per Classe	24
4.3.2 Distribuzione delle Probabilità Massime	25
4.3.3 Distribuzione delle occorrenze per colonna	25
4.4 Conclusioni	26

1. Introduzione

Lo studio del pitting negli ingranaggi rappresenta un aspetto molto importante nell'analisi dell'affidabilità e della manutenzione predittiva dei sistemi meccanici. In questo capitolo si introduce il dataset utilizzato per la diagnosi del pitting su un cambio monostadio, descrivendone la struttura, le condizioni operative, le caratteristiche dei segnali acquisiti e la metodologia di campionamento.

1.1 Descrizione del Dataset

Il dataset è composto da numerosi file di testo (.txt) raccolti durante esperimenti di pitting su un cambio monostadio. Ogni file contiene informazioni sulle vibrazioni e sul segnale tachometrico, ed è organizzato in quattro colonne:

- **Colonna 1:** Accelerazione orizzontale.
- **Colonna 2:** Accelerazione assiale.
- **Colonna 3:** Accelerazione verticale.
- **Colonna 4:** Segnale del tachimetro.

I segnali sono acquisiti in modo sincrono per garantire una corretta analisi delle vibrazioni e delle anomalie associate al degrado degli ingranaggi.

1.2 Condizioni Operative e Denominazione dei File

I file sono denominati secondo le condizioni operative, indicando la velocità di rotazione in giri/min e la coppia applicata in Nm. Ad esempio, un file denominato V100_200N_2.txt rappresenta dati acquisiti con 100 giri/min in ingresso e 200 Nm di coppia sull'albero di uscita, dove il suffisso “_2” indica la seconda ripetizione dell'esperimento.

1.3 Caratteristiche del Segnale di Vibrazione

I segnali di vibrazione sono acquisiti con una frequenza di campionamento di $20480Hz$, valore scelto per catturare in dettaglio le componenti dinamiche rilevanti, rispettando il teorema di Nyquist. Il tachimetro, montato sull'albero di uscita, fornisce un impulso per ogni rotazione, consentendo la segmentazione precisa del segnale.

1.4 Durata dell'Acquisizione e Segmentazione

La durata dell'acquisizione varia in funzione della velocità di rotazione:

- **100-200 rpm:** circa 12 secondi di dati.
- **300-1000 rpm:** circa 6 secondi.
- **1200 rpm e oltre:** circa 3 secondi.

NOTA Per garantire un bilanciamento equo dei dati di addestramento ed evitare squilibri nei pesi dei campioni, è stata sviluppata la funzione `processFilesForSegmentation` (file `processFilesForSegmentation.m`), la quale ha elaborato e memorizzato segmenti uniformi di 3 secondi per ciascun campione.

Nel caso si volesse riprodurre il codice costruito, è necessario commentare la parte di codice che richiama la suddetta funzione dopo la prima esecuzione. Di seguito il codice:

```
...
%% Preparazione dei dati
% Per processare i file di training:
% !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
% ATTENZIONE: I file di training devono essere processati una sola volta.
% COMMENTARLI DOPO LA PRIMA ESECUZIONE DI QUESTO SCRIPT O DI complete_cnn_sum.m
% !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

processFilesForSegmentation('B - PHM America 2023 -
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_0 (Healthy)\')
;
processFilesForSegmentation('B - PHM America 2023 -
```

```

Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_1\');
processFilesForSegmentation('B - PHM America 2023 -
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_2\');
processFilesForSegmentation('B - PHM America 2023 -
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_3\');
processFilesForSegmentation('B - PHM America 2023 -
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_4\');
processFilesForSegmentation('B - PHM America 2023 -
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_6\');
processFilesForSegmentation('B - PHM America 2023 -
Dataset\Data_Challenge_PHM2023_training_data\Pitting_degradation_level_8\');

% Per processare i file di test:
processFilesForSegmentation('B - PHM America 2023 -
Dataset\Data_Challenge_PHM2023_test_data\');

% Per processare i file di validation:
processFilesForSegmentation('B - PHM America 2023 -
Dataset\Data_Challenge_PHM2023_validation_data\');
...

```

1.5 Dati di Degradazione e Livelli di Salute

Gli ingranaggi sono stati sottoposti a pitting controllato, con livelli di degradazione variabili da 0 (sano) a 10 (massima degradazione). Il dataset di training comprende 78 condizioni operative distribuite su 7 livelli di salute, con alcune esclusioni (pitting 5, 7, 9, 10) per l'addestramento. Ogni condizione è registrata con 5 ripetizioni, per un totale di 2016 file.

1.6 Banco di Prova e Strumentazione

L'esperimento è condotto su un cambio monostadio con ingranaggi cilindrici:

- **Ingranaggio di comando:** 40 denti.
- **Ingranaggio condotto:** 72 denti.

Il banco di prova utilizza due servomotori *Siemens* da $45kW$ (uno per l'azionamento e uno per il carico) e un accelerometro triassiale fissato sulla scatola del cambio vicino al cuscinetto di uscita.

1.7 Obiettivi dell'Analisi

L'obiettivo del problema è sviluppare un modello in grado di stimare la gravità del guasto a partire da dati raccolti in condizioni operative eterogenee. In particolare, il modello deve generalizzare su livelli di guasto e condizioni operative non presenti nel set di addestramento, poiché i dati di training coprono solo uno stato sano e sei livelli di guasto, mentre i set di test e validazione comprendono undici livelli di salute. Inoltre, il sistema deve fornire una misura di fiducia nelle sue previsioni, in modo da poter valutare l'attendibilità dei risultati prima di procedere con azioni successive.

Nei capitoli successivi verranno approfondite le tecniche di analisi e i risultati ottenuti.

1.8 Procedure utilizzate

Nel presente lavoro sono stati sviluppati due approcci distinti per la rilevazione di anomalie e la stima del fault level nei segnali vibratorii.

Il primo approccio si basa sull'impiego di un **autoencoder** convoluzionale (CNN) addestrato in modalità *unsupervised* utilizzando esclusivamente dati appartenenti alla condizione *healthy*. In questo contesto, il modello apprende a ricostruire accuratamente i segnali normali, e durante la fase di testing, l'errore di ricostruzione viene impiegato come indicatore per rilevare eventuali anomalie. Se tale errore supera una soglia prestabilita — definita come la media degli errori sui segnali *healthy* sommata a tre volte la loro deviazione standard — il segmento viene considerato anomalo e assegnato a una classe di guasto¹, proporzionale alla deviazione osservata. Per ogni segmento, inoltre, viene generata una distribuzione di probabilità (mediante una funzione gaussiana) dalla quale si ricava anche un valore di confidenza binaria.

Il secondo approccio adotta una strategia differente: una CNN viene addestrata in modalità *supervised* per classificare i segnali, opportunamente convertiti in

¹Le classi di guasto rappresentano i diversi livelli di degrado del motore.

immagini, e successivamente viene integrato un modulo di **anomaly detection** basato su *one-class SVM*. In questo schema, la CNN ha il compito di estrarre feature significative dai segnali; tali feature, prelevate da uno dei layer intermedi, vengono poi utilizzate per addestrare modelli *one-class SVM* specifici per le classi base (ad esempio, “4”, “6” e “8”). Durante il testing, se un segmento viene classificato in una classe base presente nel training, ma il punteggio di anomalia risulta inferiore a una soglia definita, e la classe immediatamente successiva è una di quelle assenti nel training, il sistema riassegna il campione a tale classe mancante (ad esempio, da “4” a “5”).

Per l’analisi delle feature estratte nelle diverse fasi del processo, sono state utilizzate funzioni specifiche in combinazione con l’applicazione *Diagnostic Feature Designer*. Questo strumento mette a disposizione un’interfaccia grafica versatile, pensata per supportare la progettazione e la validazione delle caratteristiche all’interno dei flussi di lavoro per la manutenzione predittiva. Grazie alle sue funzionalità interattive, è possibile esplorare, confrontare e raffinare le feature in modo semplice ed efficace.

Tra le principali funzioni impiegate figura `datatable` (file `datatable.m`), utilizzata per generare una tabella unificata a partire da una cell array di strutture. Nel caso specifico, questa tabella include informazioni come le velocità del motore lungo gli assi X , Y e Z , gli `rpm` e la coppia motrice. Questi ultimi due valori non sono contenuti direttamente nei dati, ma vengono estratti dal nome dei file di input in formato `.txt` (ad esempio, dal file `V900_50N_2.txt` si ricavano rispettivamente 900 `rpm` e 50 `N`).

Una volta completata la fase di estrazione, l’applicazione può essere avviata tramite il comando: `diagnosticFeatureDesigner`.

NOTA In entrambi gli approcci, i file utilizzati sono stati scaricati dai seguenti link: https://phm-datasets.s3.amazonaws.com/Data_Challenge_PHM2023_training_data.zip per i dati di training, https://phm-datasets.s3.amazonaws.com/Data_Challenge_PHM2023_test_data.zip per i dati di testing e https://phm-datasets.s3.amazonaws.com/Data_Challenge_PHM2023_validation_data.zip per i dati di validation. Una volta scaricati, sono stati estratti e salvati all’interno della cartella `B - PHM America 2023 - Dataset`, nello stesso percorso dei file `MATLAB`.

Nel caso si volesse riprodurre il seguente progetto, è imperativo seguire la stessa procedura.

1.9 Repository

LA repository del seguente progetto è presente al seguente link: <https://github.com/GabrielPiercecchi/ProgettoManutenzionePreventivaR-AI>. Per scaricare il codice è sufficiente cliccare sul button `<> Code` e successivamente su "Download ZIP".

2. Addestramento CNN-AUTOENCODER

2.1 Introduzione

Il modello che andremo a descrivere in questo capitolo, rappresenta una pipeline completa per l'anomaly detection e la stima del fault level, espresso su una scala da 0 a 10. L'approccio sfrutta un **autoencoder CNN** in modalità unsupervised, combinato con una semplice estrazione delle feature. L'obiettivo principale è sviluppare un modello in grado di apprendere il comportamento "sano" (healthy) e, successivamente, riconoscere e quantificare le anomalie nei dati raccolti in condizioni operative eterogenee.

```
Il nome del file relativo a questo modello è  
complete_autoencoder_CNN_pipeline.m
```

2.2 Procedimento

2.2.1 STEP 1 - Caricamento e Pre-elaborazione dei Dati

Il processo inizia con il caricamento dei dati "healthy", ossia quelli relativi al livello 0 di pitting, prelevati da una cartella specifica. Una funzione filtra e carica i file in formato .txt, registrando un totale di 287 file healthy. Su questi dati viene applicata una dummy feature extraction che non altera il contenuto originale, seguita dalla segmentazione dei segnali in finestre di 3 secondi. La funzione per la segmentazione suddivide ciascun file in segmenti uniformi, ottenendo esattamente 287 segmenti, così da garantire la compatibilità con l'autoencoder CNN.

2.2.2 STEP 2 - Addestramento dell'Autoencoder CNN

A seguito della pre-elaborazione, viene definita l'architettura dell'autoencoder CNN. Il layer di input è definito con dimensioni pari a [samplesPerSeg, numChannels,

1], dove `samplesPerSeg` indica il numero di campioni per segmento (20480) e `numChannels` il numero di canali (ad esempio, `acc_x`, `acc_y`, `acc_z` e `tachimetro`).

L'encoder, costituito da strati convoluzionali, ReLU e pooling, riduce la dimensionalità del segnale, mentre il decoder utilizza strati di convoluzione trasposta per ricostruire l'input.

Il modello viene addestrato in modalità *unsupervised* utilizzando i dati *healthy* come riferimento; durante il training, MATLAB mostra la finestra di "training progress" (Figura 2.1).

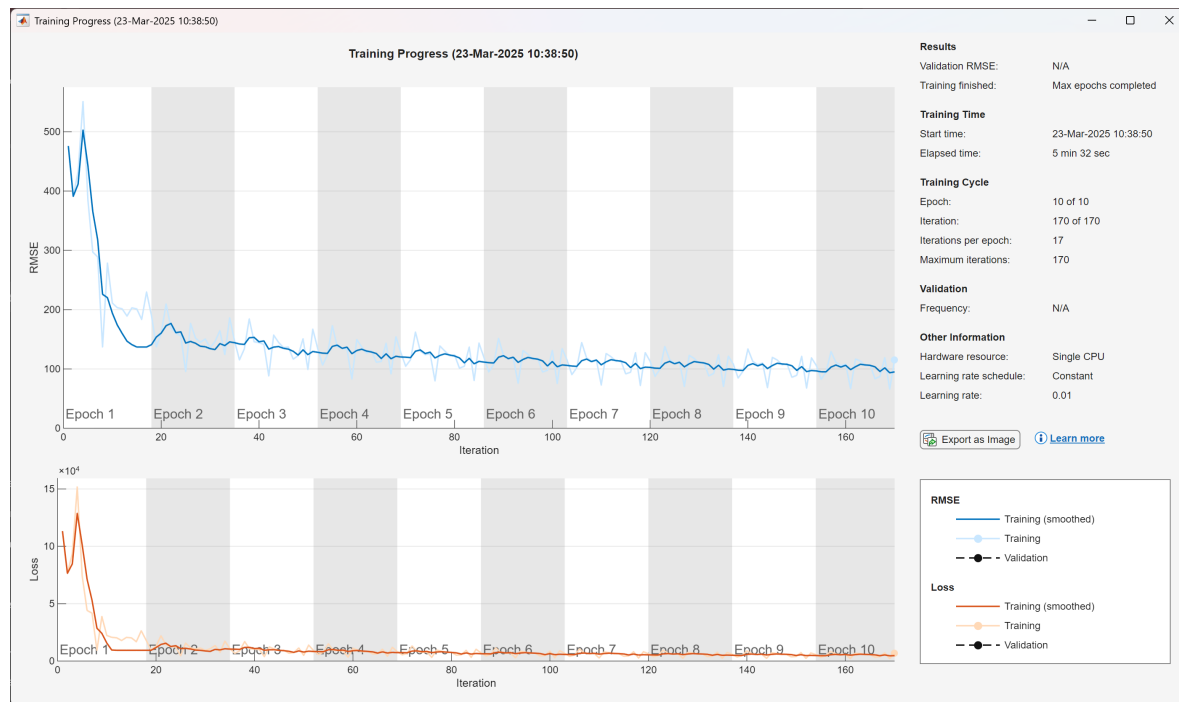


Figura 2.1: Training Progress

Il training è stato condotto su singola CPU e, dopo circa 10 epoche, il modello ha completato l'addestramento con una soglia di errore *healthy* pari a 0.4573.

Dopo l'addestramento, autoencoder viene utilizzato per ricostruire i dati *healthy*. L'errore di ricostruzione viene quindi calcolato come la media degli errori quadrati (MSE) tra ogni segmento di input e il corrispondente output ricostruito. Successivamente, viene generato un istogramma per visualizzare la distribuzione dell'errore di ricostruzione *healthy* (Figura 2.2). Sull'asse delle ascisse sono riportati i *mini-batch*, ovvero i sottoinsiemi di dati *healthy* forniti progressivamente alla

rete durante l'apprendimento. Sull'asse delle ordinate è rappresentata la *loss*, cioè la misura dell'errore tra i dati di input e quelli ricostruiti dall'autoencoder.

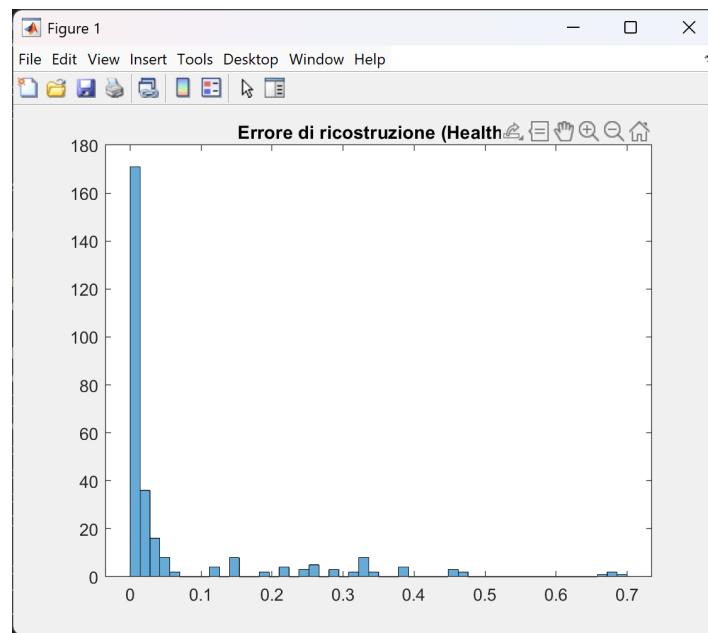


Figura 2.2: Istogramma dell'errore di ricostruzione sui dati healthy

Infine, la soglia di errore, utile per distinguere in seguito i dati anomali, viene impostata come la media dell'errore di ricostruzione più tre volte la deviazione standard.

2.2.3 STEP 3 - Caricamento dei dati unhealthy

Al termine della fase di addestramento, il modello viene applicato ai dati *unhealthy*, corrispondenti ai livelli di degrado 1, 2, 3, 4, 6 e 8. In questa fase, vengono estratte le feature dai dati degradati, seguendo un approccio analogo a quello utilizzato per i dati *healthy*. In questo primo tentativo, l'estrazione avviene in modalità *dummy*: i segnali vengono semplicemente processati tramite la funzione preposta, senza l'applicazione di trasformazioni complesse.

Successivamente, i dati *unhealthy* vengono segmentati in finestre temporali, al fine di generare gli input necessari per le elaborazioni successive.

2.2.4 STEP 4 - Valutazione degli Errori e Mapping in Livello

L'autoencoder, precedentemente addestrato sui dati *healthy*, calcola l'errore di ricostruzione per ciascun segmento, definito come la differenza quadratica media tra input e output. Se tale errore supera una soglia stabilita per i dati *healthy*, pari a 0,4573, il segmento viene considerato anomalo e mappato su un livello di guasto (*fault level*) proporzionale alla deviazione. In questo caso, il livello medio di guasto predetto risulta essere circa 0,01, con il 100% dei segmenti classificati con alta confidenza.

Il processo di mapping in un determinato livello di degradazione si basa sul confronto tra l'errore di ricostruzione del segmento e una soglia statistica, calcolata come la media degli errori dei segnali *healthy* più tre volte la relativa deviazione standard. Quando l'errore di un segmento eccede tale soglia, esso viene etichettato come anomalo, e gli viene assegnato un livello di guasto proporzionale allo scostamento rispetto alla soglia stessa.

ESEMPIO: Supponiamo che il sistema sia stato configurato con una soglia di accettabilità pari a 0,4573. Questo implica che, se l'errore di ricostruzione di un segmento supera tale soglia, il segmento viene considerato anomalo.

La gravità dell'anomalia viene successivamente quantificata calcolando lo scostamento dell'errore rispetto alla soglia, espresso in unità di deviazione standard degli errori calcolati su dati sani. Il valore ottenuto viene passato alla funzione `mapErrorToSeverity`, che assegna un livello discreto di severità del guasto, generalmente compreso tra 0 e 10, in base all'entità dello scostamento.

Ad esempio, se l'errore supera la soglia di circa 2 deviazioni standard, il segmento potrebbe essere associato a un livello di severità pari a 7 su 10. A partire da questo valore, si costruisce una distribuzione di probabilità utilizzando una funzione gaussiana centrata sul livello stimato.

Da tale distribuzione viene infine derivata una misura di confidenza binaria tramite la funzione `computeConfidence`. Se lo scostamento risulta sufficientemente elevato (ad esempio maggiore di 1 o 1,5 deviazioni standard), la confidenza viene fissata al valore massimo, ovvero 1 (alta confidenza).

Per ogni segmento, viene inoltre generata una distribuzione di probabilità tramite una funzione gaussiana centrata sul livello predetto, da cui si ricava una mi-

sura di confidenza binaria. Questa procedura è rappresentata in Figura 2.3, in cui l'istogramma dell'errore di ricostruzione sui dati *unhealthy* evidenzia chiaramente la posizione della soglia.

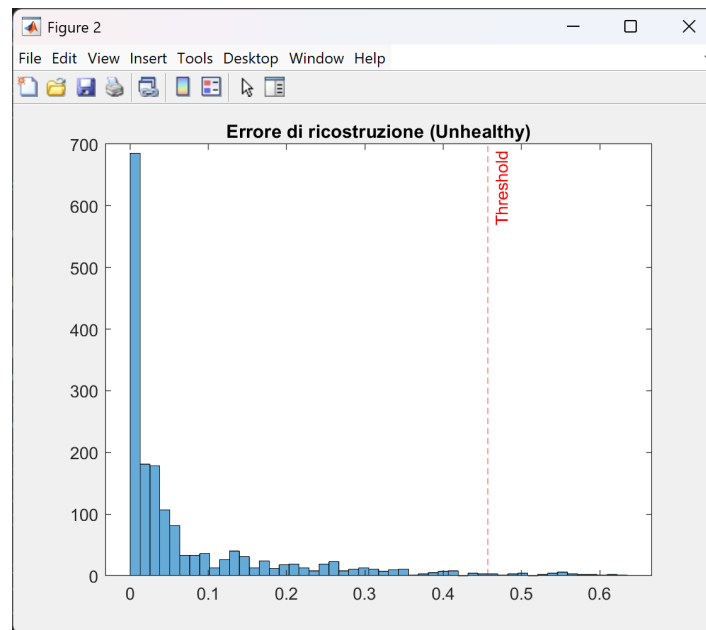


Figura 2.3: Istogramma dell'errore di ricostruzione sui dati *unhealthy*, con la linea della soglia evidenziata

2.2.5 STEP 5 - Analisi dei Meta-dati Operativi

Si procede analizzando i meta-dati operativi, come la velocità del motore e il torque, per valutare la correlazione con l'errore medio di ricostruzione a livello di file. I grafici a dispersione mostrano chiaramente:

- In 800 file test (segmentati in 800 segmenti) il mapping e la predizione con l'autoencoder risultano analoghi a quelli dei dati *unhealthy*, con livelli medi predetti pari a 0.01 e confidenza elevata (100%).
- Le relazioni tra Motor Speed, Torque ed errore medio sono illustrate nelle Figure 2.4, 2.5 e 2.6, fornendo una visione integrata delle condizioni operative.

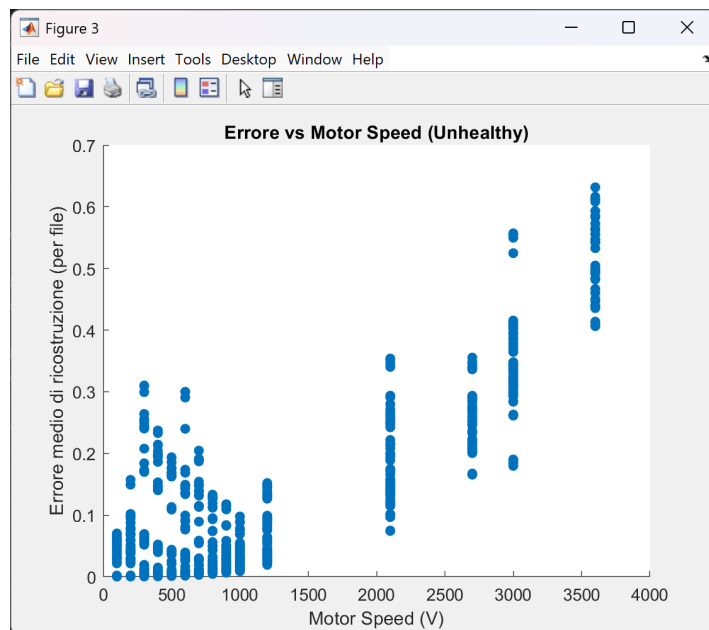


Figura 2.4: Scatter plot “Errore vs Motor Speed (Unhealthy)”, che mostra la relazione tra velocità del motore e errore medio di ricostruzione

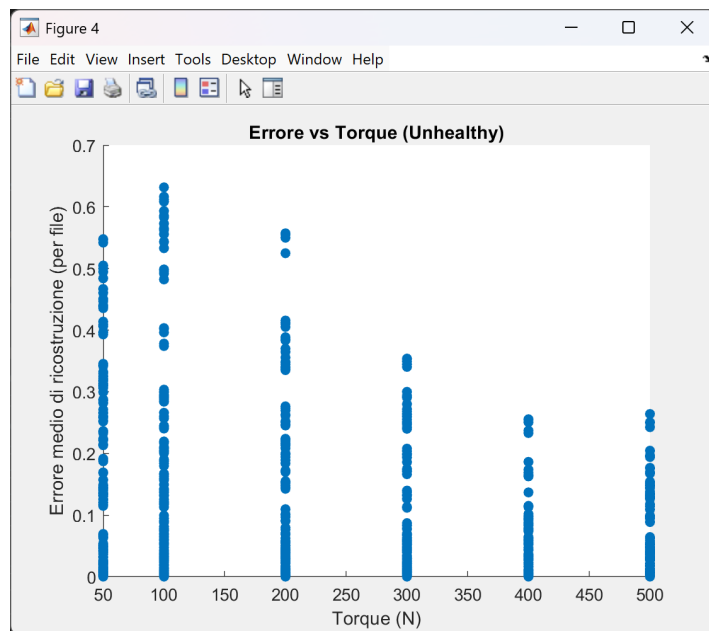


Figura 2.5: Scatter plot “Errore vs Torque (Unhealthy)”, per evidenziare la correlazione tra torque e errore di ricostruzione

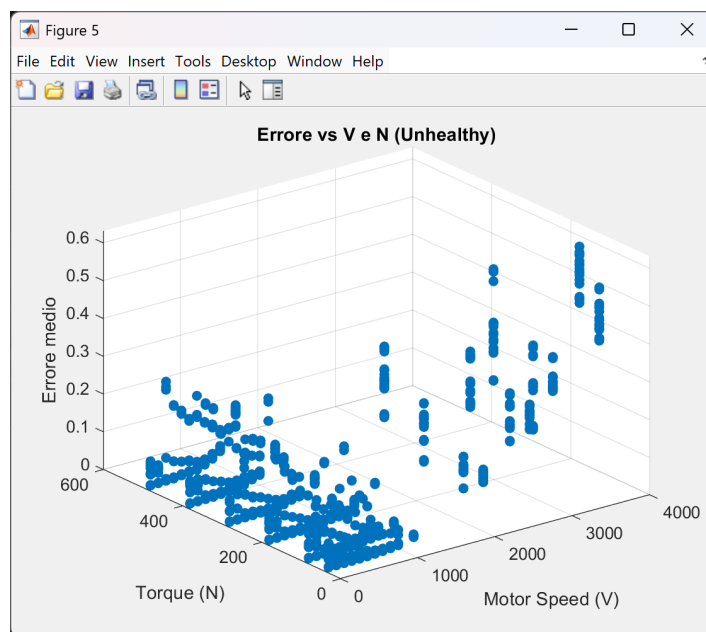


Figura 2.6: Scatter plot 3D “Errore vs V e N (Unhealthy)”, che offre una visione integrata di come velocità e torque interagiscano nell’influencare l’errore medio

2.2.6 STEP 6 - Generazione dei File di Submission per i Dati Unhealthy

Nel sesto step della pipeline, il sistema raccoglie e organizza le informazioni prodotte dal modello relative all’analisi dei dati **unhealthy** — tra cui l’identificativo del segmento, la distribuzione di probabilità sui livelli di degradazione e la confidenza della previsione — all’interno di una tabella. Quest’ultima viene successivamente esportata in un file CSV denominato *submission_unhealthy.csv*. Il file generato costituisce l’output finale di questa fase della pipeline, pronto per essere impiegato nelle successive attività di valutazione del modello o nell’analisi comparativa dei risultati ottenuti.

2.2.7 STEP 7 - Caricamento dei dati di test e generazione del submission_test

Il modello viene successivamente applicato ai dataset di test. In questa fase sono stati caricati 800 file che, dopo la segmentazione e la fase di predizione, hanno prodotto un livello medio di severità pari a 0,01, calcolato a partire dagli errori di ricostruzione dei singoli segmenti. Tale valore rappresenta una stima numerica

della gravità del degrado nei dati analizzati e, risultando estremamente basso, indica che la quasi totalità dei segmenti è stata riconosciuta come appartenente a condizioni sane. Inoltre, la confidenza associata a queste predizioni è pari al 100%, a conferma del fatto che il modello ha operato con elevata sicurezza, grazie a errori di ricostruzione significativamente inferiori alla soglia definita per discriminare tra stato sano e deteriorato.

Una volta ottenuti i segmenti di test, il modello addestrato viene utilizzato per ricostruire ciascun segmento. La differenza tra il segnale originale e quello ricostruito viene quantificata attraverso il *Mean Squared Error (MSE)* calcolato per ogni segmento, producendo un vettore di errori di ricostruzione. La distribuzione di tali errori viene infine rappresentata tramite un istogramma (Figura 2.7), arricchito da una linea rossa che indica la soglia precedentemente definita, utile a evidenziare i segmenti che superano il limite di errore ritenuto accettabile.

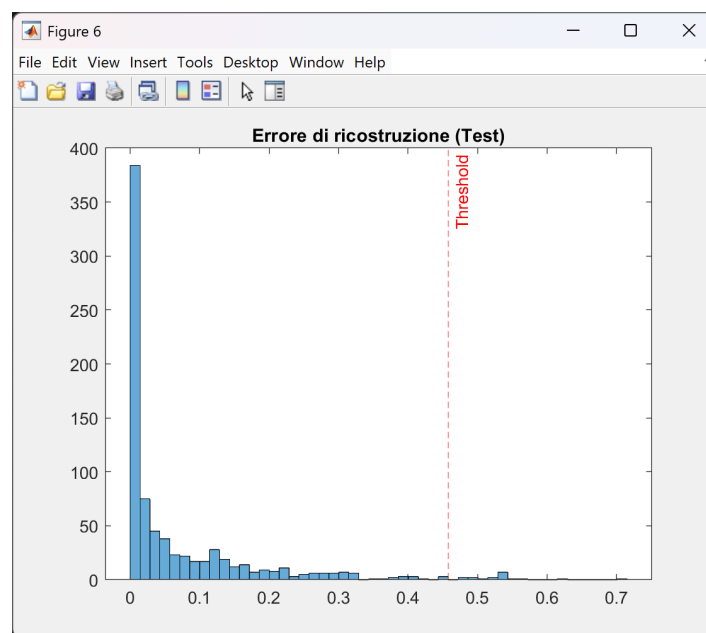


Figura 2.7: Istogramma dell'errore di ricostruzione sui dati di test

I risultati ottenuti vengono organizzati all'interno di una matrice che, per ciascun segmento, contiene un identificativo univoco, la distribuzione di probabilità sui diversi livelli di degradazione e il valore di confidenza associato. Questa matrice viene successivamente convertita in una tabella MATLAB, a cui vengono assegnati nomi di colonna descrittivi, come ad esempio 'sample_id', 'prob_0', ..., 'prob_10', 'confidence'.

Il passo conclusivo consiste nell'esportazione della tabella in un file CSV, denominato `submission_cnn.csv`, che rappresenta il prodotto finale della fase di testing e può essere utilizzato per la valutazione del modello.

Procedimento analogo con i dati presenti nella cartella *validation* (STEP 7.1), con la tabella finale esportata nel file CSV denominato `submission_validation_cnn.csv`.

2.2.8 STEP 8 - Tuning e Analisi dei Threshold

In questa sezione vengono eseguite operazioni di tuning e analisi, con l'obiettivo di esplorare soglie alternative per l'errore di ricostruzione sui dati healthy e di raccogliere statistiche chiave. Questo processo aiuta a comprendere meglio la distribuzione degli errori e a valutare come differenti criteri di soglia possano influenzare l'identificazione delle anomalie.

L'analisi riporta che:

- La media degli errori healthy è pari a 0.0665, con una deviazione standard di 0.1303. Tuttavia, la soglia scelta per identificare anomalie ($media + 3 \times std = 0.4573$) risulta relativamente alta rispetto alla media, il che potrebbe portare il modello a considerare "normali" anche alcuni segmenti con un errore elevato, riducendo la sensibilità nella rilevazione dei guasti lievi.
- Sono state esplorate soglie alternative, rispettivamente $media + 2 \times std = 0.3270$ e $media + 4 \times std = 0.5876$, per valutare il bilanciamento tra falsi positivi e falsi negativi.

Queste soglie vengono sovrapposte sull'istogramma degli errori healthy (Figura 2.8), evidenziando il bilanciamento tra falsi positivi e falsi negativi.

Alla fine, le feature estratte dai dati healthy e unhealthy vengono combinate e salvate in un file MAT.

2.3 Osservazioni Finali

Nonostante l'analisi delle soglie, il metodo presenta alcune limitazioni. Una delle principali criticità riguarda la **scarsa generalizzazione sui dati unhealthy**: il modello è stato addestrato esclusivamente su dati healthy, il che potrebbe impedirgli di

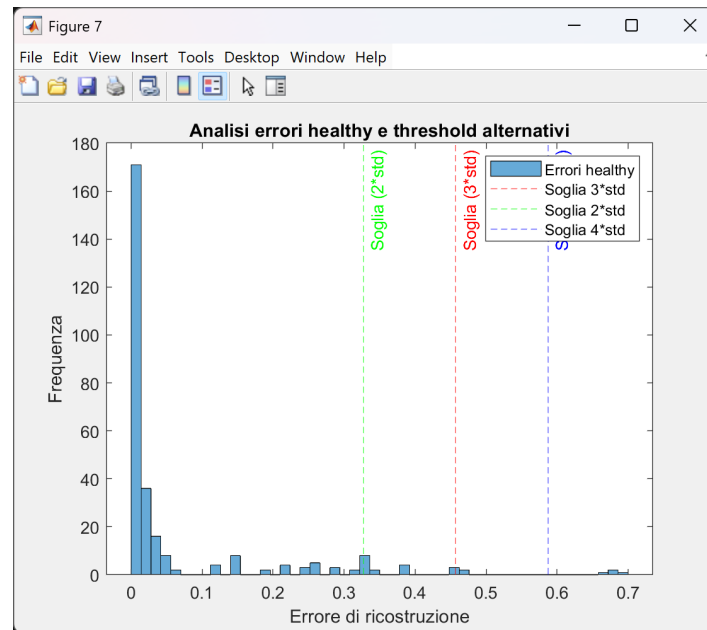


Figura 2.8: Istogramma degli errori healthy con le soglie alternative ($2 \times \text{std}$, $3 \times \text{std}$ e $4 \times \text{std}$) evidenziate

catturare correttamente la distribuzione degli errori nei dati guasti. Di conseguenza, la soglia scelta potrebbe non separare in modo efficace le condizioni normali da quelle anomale, portando a errori di classificazione.

Infine, un'ulteriore limitazione è la **mancanza di validazione** con dati etichettati unhealthy. Non essendo stata effettuata una verifica diretta su dati con etichette di guasto, non è possibile determinare con precisione il tasso di falsi positivi e falsi negativi. Questo rende difficile valutare l'effettiva efficacia del modello nel distinguere tra healthy e unhealthy, suggerendo la necessità di un approccio più robusto per migliorare l'accuratezza delle previsioni.

Questo ha comportato l'abbandono di questo modello in favore di uno nuovo che fosse più preciso nelle previsioni.

3. Addestramento CNN con SVM

3.1 Introduzione

In questo capitolo viene presentato un approccio alternativo per la stima della gravità dei guasti e il rilevamento delle anomalie. In questa metodologia, ciascun segmento di dati viene trattato come un'immagine tridimensionale di dimensioni $[20000 \times 3 \times 1]$.

I file di input, forniti in formato testo, contengono quattro colonne, ma vengono utilizzate esclusivamente le prime tre, corrispondenti alle coordinate spaziali X , Y e Z .

Il modulo implementato adotta una strategia di *anomaly detection*, che interviene in particolare sulla classe "2" e sulle classi di base (4, 6, 8). Quando un'anomalia viene rilevata all'interno di una di queste classi, il sistema provvede a riassegnare il segmento alla corrispondente classe successiva mancante nel training (5, 7, 9), al fine di ottenere una rappresentazione più accurata delle condizioni di guasto nel dataset.

Il nome del file relativo a questo modello è `complete_cnn_svm.m`

3.2 Parametri e Configurazione

Inizialmente vengono definiti i parametri fondamentali per l'elaborazione e l'addestramento di un modello di classificazione. La frequenza di campionamento è impostata a 20480 Hz, in modo che ogni secondo di segnale contenga 20480 campioni, dato che la durata di ciascun segmento è fissata a 1 secondo. Il numero di campioni per segmento viene calcolato come il prodotto tra la frequenza di campionamento e la durata del segmento.

Il problema di classificazione viene formulato su 7 classi, ottenute rietichettando le classi originali (0, 1, 2, 3, 4, 6, 8) in modo che risultino consecutive. Sono inoltre definiti il batch size (32) e il numero di epoche (15) per il processo di ottimizzazione. Infine, vengono specificati i percorsi per i dati di training e di test e

viene creata una mappa (`containers.Map`) per associare i nomi delle cartelle alle rispettive etichette numeriche.

3.3 Caricamento e Segmentazione dei Dati

Per la segmentazione, una funzione riceve in ingresso il percorso della cartella contenente i file di test e la lunghezza di ciascun segmento (in campioni). Il codice individua tutti i file di testo presenti nella directory e, per ognuno di essi, tenta di caricare i dati. Se un file contiene almeno tre colonne, vengono selezionate le prime tre; altrimenti, viene generato un errore che interrompe l'elaborazione del file.

Per ogni file, viene calcolato il numero di segmenti ottenibili dividendo il numero totale di campioni per la lunghezza del segmento, escludendo quelli incompleti. Ogni segmento, estratto come intervallo di dati, viene concatenato lungo la terza dimensione di un array e, parallelamente, viene memorizzato il nome del file di origine in un cell array.

Al termine, la funzione restituisce un array tridimensionale organizzato come `[segmentLength x 3 x numSegments]` insieme alle informazioni sui file.

3.3.1 Preprocessing e Normalizzazione

I dati di training vengono normalizzati canale per canale: per ciascuna delle tre colonne vengono calcolate la media e la deviazione standard, valori poi utilizzati per standardizzare sia i dati di training che quelli di test. Successivamente, i dati, originariamente organizzati in array 3D, vengono convertiti in formato 4D (`[segmentLength, 3, 1, N]`) per garantire la compatibilità con la rete neurale convoluzionale. Le etichette di training sono, infine, convertite in variabili categoriche per facilitare il processo di apprendimento supervisionato.

3.3.2 Suddivisione Training/Validation e Definizione del Modello CNN

Il dataset di training viene suddiviso in un set per l'addestramento (80%) e uno per la validazione (20%) mediante la funzione `cvpartition`. Il modello CNN viene definito attraverso una serie di layer:

- Un `imageInputLayer` per l'input, seguito da strati di convoluzione, attivazione (ReLU) per introdurre non linearità e pooling che riducono la dimensionalità spaziale dei dati, facilitando l'estrazione delle caratteristiche più rilevanti;
- Layer completamente connessi e un `dropoutLayer` per ridurre l'overfitting;
- Un layer softmax e un `classificationLayer` per la classificazione finale.

L'addestramento della rete avviene in modalità supervisionata, utilizzando l'ottimizzatore `RMSprop`, un tasso di apprendimento iniziale predefinito, un numero specifico di epoche e una dimensione dei mini-batch fissata, con il monitoraggio delle prestazioni sul validation set.

3.3.3 Visualizzazione e Valutazione del Modello

Durante l'addestramento, le feature interne vengono estratte dal layer `'relu_fc1'` utilizzando la funzione `activations`. Successivamente, il modello viene valutato sul validation set tramite la funzione `classify`, si calcola l'accuratezza e si genera una matrice di confusione (tramite `confusionchart`), che viene salvata per ulteriori analisi, mostrata in Figura 4.2.

3.3.4 Predizione sul Test Dataset

Il modello viene utilizzato per predire le etichette dei segmenti del test set. Utilizzando il comando `classify`, vengono ottenute le etichette predette, mentre con `predict` vengono calcolate le probabilità associate a ciascuna classe.

Per ogni segmento, viene identificata la classe con la probabilità massima (confidenza) e vengono stampati a video il numero del segmento, il nome del file di origine e il livello di guasto predetto, insieme al relativo valore di confidenza. Questo processo fornisce una panoramica dettagliata e interpretativa delle performance del modello sui dati di test.

Un procedimento analogo viene eseguito per la predizione sul Validation Dataset.

3.4 Modulo di Anomaly Detection Esteso

Per aumentare la robustezza del sistema, è stato implementato un modulo di **anomaly detection esteso**, che introduce un controllo aggiuntivo sulle cosiddette classi "base". In particolare, l'attenzione è rivolta alle classi "4", "6" e "8", considerate come livelli di riferimento. Qualora venga rilevato un comportamento anomalo in uno di questi segmenti, esso viene riassegnato alla corrispondente classe mancante immediatamente successiva — rispettivamente "5", "7" e "9" — che non sono presenti nel training set, ma servono a migliorare la rappresentazione delle condizioni di guasto nel dataset.

Il processo inizia con la definizione delle classi di base e la creazione di una mappa (`containers.Map`) che associa ciascuna classe di base alla corrispondente classe mancante. Successivamente, le feature vengono estratte dai dati di training dal layer 'relu_fc1, utilizzando la funzione `activations`. Queste rappresentazioni interne vengono quindi utilizzate per addestrare dei modelli **One-Class Support Vector Machine (SVM)** tramite la funzione `fitcsvm`, con un kernel gaussiano. I modelli sono configurati per standardizzare i dati e rilevare una frazione di outlier pari al 5%, così da apprendere il comportamento "normale" per ciascuna classe di base.

NOTA: Nel dettaglio, durante la fase di addestramento, i dati vengono utilizzati per costruire il modello di classificazione. Il primo passo consiste nel calcolare la media e la deviazione standard per ciascuna caratteristica (feature) dei dati di training. Questi valori vengono utilizzati per standardizzare i dati: ogni valore di una caratteristica viene sottratto dalla sua media e poi diviso per la deviazione standard. Nella fase di test, la standardizzazione dei dati di test avviene nello stesso modo, utilizzando tuttavia la media e la deviazione standard calcolate sui dati di training, anziché sui dati di test stessi.

Proseguendo, durante la fase di test, le feature vengono estratte allo stesso modo e, per ogni segmento inizialmente classificato come appartenente a una classe base, il modello SVM calcola uno score. Se tale score è inferiore a una soglia predefinita (in questo caso 0.7), il segmento viene considerato anomalo e la sua etichetta viene riassegnata alla classe mancante corrispondente, secondo la mappa definita.

3.4.1 Calcolo classe predetta

In generale, se la classe predetta non è una classe mancante (5, 7 o 9), si individua il suo indice tra le classi viste e si assegna una probabilità pari a 1 nella colonna corrispondente della matrice. Se invece appartiene alle classi mancanti, si identifica la classe base corrispondente (4 per 5, 6 per 7, 8 per 9), si estrae la sua probabilità e si ricalcola la distribuzione, trasferendo parte della probabilità alla classe mancante e ridistribuendo il resto tra le altre classi. Se la classe base non è stata vista, si assegna direttamente una probabilità pari a 1 alla classe mancante come fallback. Questo approccio garantisce una distribuzione coerente delle probabilità e migliora la gestione delle classi assenti nel modello.

3.5 Esportazione dei Risultati

Al termine del processo di classificazione e rilevamento delle anomalie, i risultati del test dataset vengono esportati in un file CSV (`submission_cnn_svm.csv`). Questo file contiene il nome del file (o l'ID del segmento), la predizione finale e il livello di confidenza associato alla predizione. È inoltre prevista un'esportazione opzionale dei risultati del validation dataset in un file separato (`validation_submission_cnn_svm.csv`), utile per ulteriori analisi e confronti. In questo modo, il sistema fornisce un output strutturato che consente di valutare le prestazioni complessive del modello e di identificare eventuali criticità nella classificazione.

4. Risultati

4.1 Addestramento

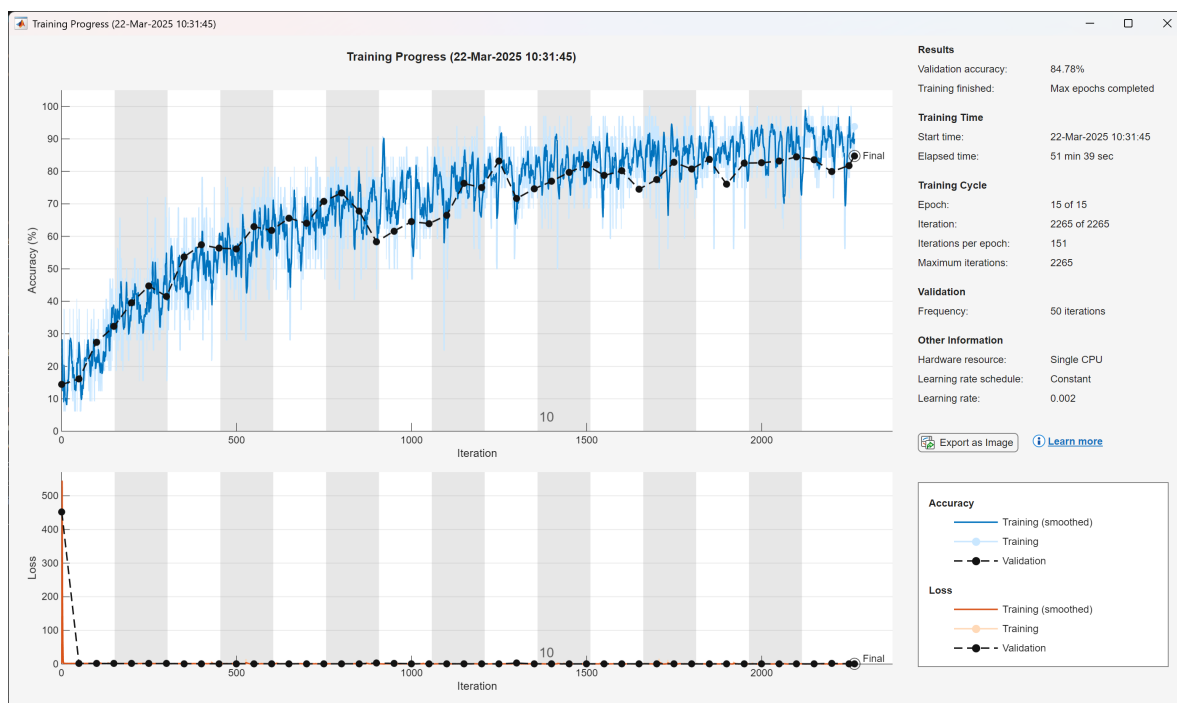


Figura 4.1: Accuracy e loss

Nel grafico mostrato nella Figura 4.1 si osserva l'andamento dell'accuratezza di training (in blu, con eventuale intervallo di oscillazione) e di validazione (in nero) in funzione delle iterazioni di addestramento. Fin dalle prime epoche, entrambe le curve mostrano una tendenza crescente, a indicare che la rete neurale convoluzionale sta gradualmente migliorando la propria capacità di riconoscere i pattern nei dati. Con il progredire delle iterazioni, l'accuratezza di training si mantiene leggermente superiore rispetto a quella di validazione, la differenza non appare eccessiva poiché l'accuratezza di validazione segue un trend analogo e si stabilizza su valori prossimi a quelli di training.

Nella parte inferiore del grafico, la funzione di costo (loss) decresce progressivamente, confermando la riduzione dell'errore complessivo man mano che la rete affina i propri parametri. L'andamento regolare mostra che il processo di ottimizzazione procede in modo efficace. Nella sezione “Results” del pannello sono riportati alcuni dati sintetici: tra questi, il valore finale di accuratezza di validazione dell'84,78%, l'epoca in cui è stato raggiunto il miglior risultato e il tempo totale di addestramento.

4.2 Confusion matrix

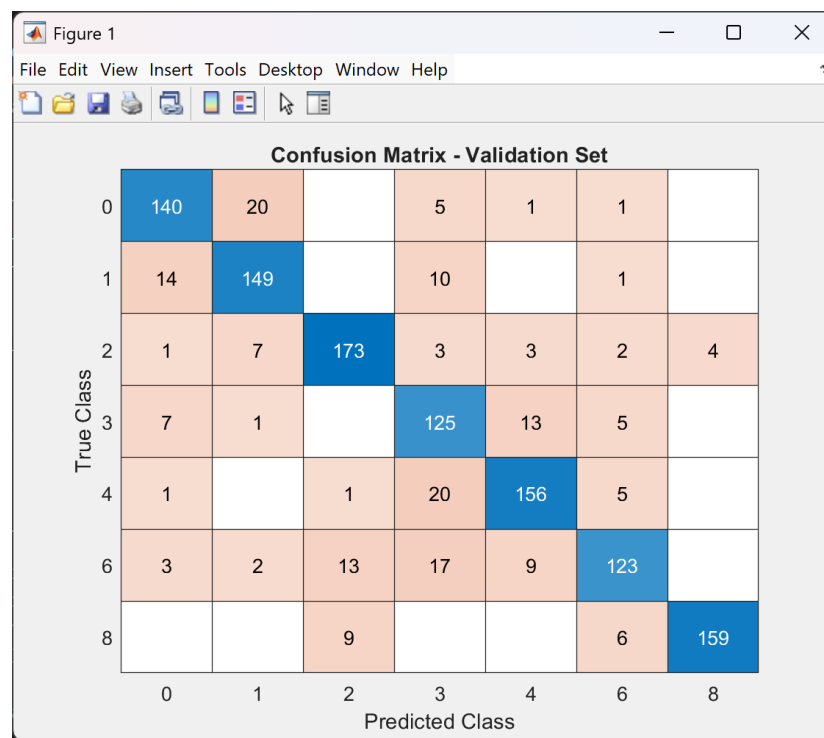


Figura 4.2: Confusion matrix

Dall'analisi della matrice di confusione, riportata in Figura 4.2, calcolata sul 20% del dataset di training, che viene denominato *validation set* per distinguerlo dal restante 80% utilizzato per l'addestramento, emerge che la maggior parte dei segmenti è stata correttamente classificata. Le righe della matrice rappresentano le classi reali, mentre le colonne indicano le classi predette dal modello. I valori lungo la diagonale principale corrispondono alle predizioni corrette, mentre quelli fuori dalla diagonale indicano errori di classificazione. Poiché i valori sulla diagonale

risultano nettamente prevalenti rispetto a quelli fuori dalla diagonale, si può concludere che il modello ha effettuato una classificazione accurata per la maggior parte dei segmenti. Nonostante alcune lievi confusioni residue, i risultati complessivi confermano che il modello, appreso durante l'addestramento, è in grado di distinguere in modo affidabile i vari livelli di gravità.

4.3 Prestazioni del modello

Per valutare le prestazioni del modello di classificazione, sono state analizzate due principali rappresentazioni grafiche:

4.3.1 Metriche di Classificazione per Classe

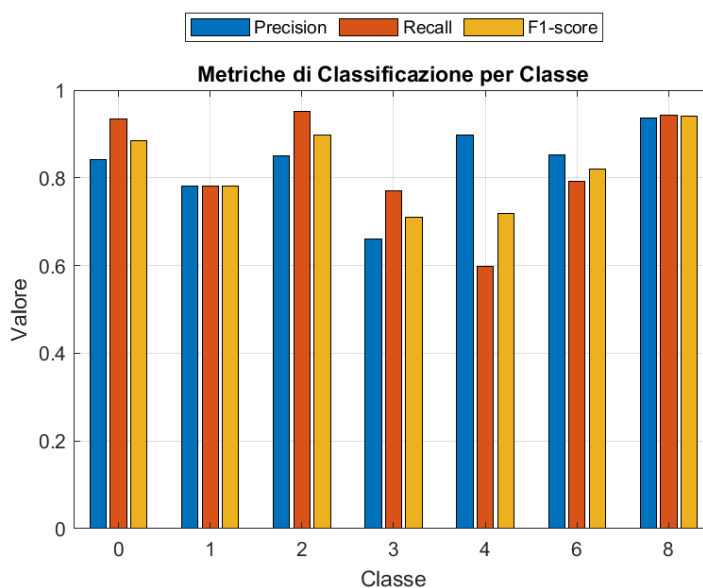


Figura 4.3: Metriche di Classificazione per Classe

Il grafico (Figura 4.3) rappresenta le metriche di *precision*, *recall* e *F1-score* per ciascuna classe. La precisione misura la frazione di predizioni corrette rispetto a tutte le predizioni effettuate per una determinata classe, il recall indica la capacità del modello di identificare correttamente gli esempi positivi, mentre l'*F1-score* bilancia questi due aspetti con una media armonica. Dai risultati emerge una buona performance generale del modello, con variazioni nelle diverse classi.

4.3.2 Distribuzione delle Probabilità Massime

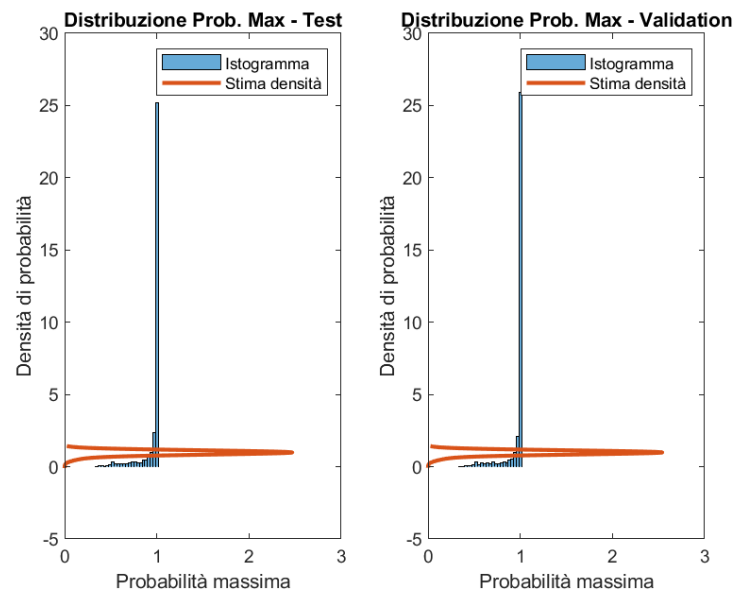


Figura 4.4: Distribuzione probabilità

Il grafico (Figura 4.4) mostra la distribuzione delle probabilità massime associate alle predizioni sia per il dataset di test che per il dataset di validazione. Quest'analisi valuta il livello di confidenza del modello nelle sue previsioni. L'istogramma e la stima della densità evidenziano un picco molto concentrato, suggerendo che il modello tende a effettuare previsioni con alta sicurezza.

4.3.3 Distribuzione delle occorrenze per colonna

Analizzando i dati forniti dalle tabelle mostrate in Figura 4.5 per i dati di test e in Figura 4.6 per i dati di validazione ottenuti dai corrispettivi file CSV, si può osservare quanto segue:

- Le colonne $prob_2$, $prob_7$ e $prob_9$ hanno il maggior numero di occorrenze come valore massimo, indicando che i livelli di guasto associati a queste probabilità sono predominanti nei dati.
- Le colonne $prob_5$, $prob_6$ e $prob_8$ presentano un numero molto basso di occorrenze, suggerendo che i livelli di guasto corrispondenti sono meno rappresentati o meno probabili.

- Le colonne $prob_0$, $prob_1$ e $prob_3$ mostrano una distribuzione intermedia, indicando che i dati sani e quelli relativi a guasti leggeri associati a queste probabilità sono presenti, ma non predominanti.
- La colonna $prob_4$ ha un numero molto basso di occorrenze, suggerendo che il livello di guasto associato a questa probabilità è raro o non significativo nei dati.

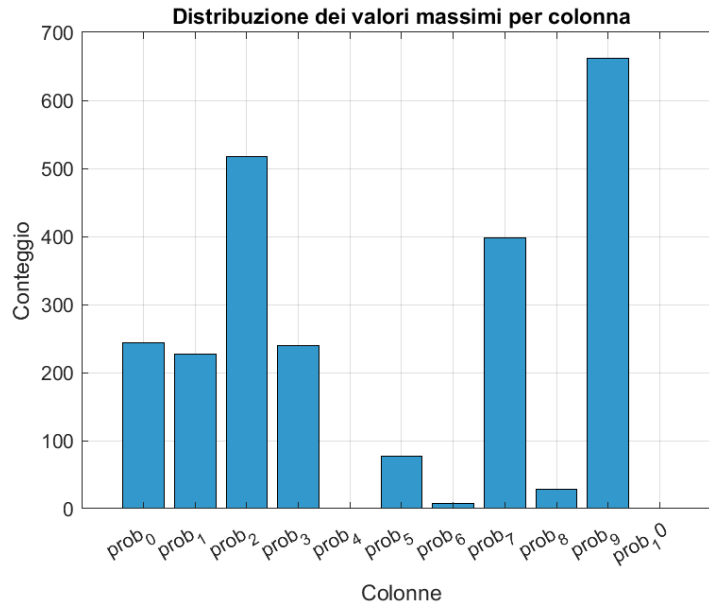


Figura 4.5: Occorrenze file submission_cnn_svm.m

Analizzando più nel dettaglio, la colonna $prob_5$, non essendo presente come livello di guasto nei dati di training, segue l'euristica del modello espressa in precedenza. È quindi probabile che contenga occorrenze identificate come guasti legati alla colonna $prob_4$, ma ritenute di gravità maggiore.

Inoltre, la colonna $prob_{10}$ è rimasta sempre vuota a causa della mancanza della colonna $prob_9$ all'interno del training.

Riassumendo, i guasti con il maggior numero di occorrenze si collocano agli estremi, risultando principalmente di basso o alto livello.

4.4 Conclusioni

I risultati ottenuti dal modello di classificazione mostrano una solida capacità di distinzione tra le diverse classi di guasto, con un'accuratezza di validazione del

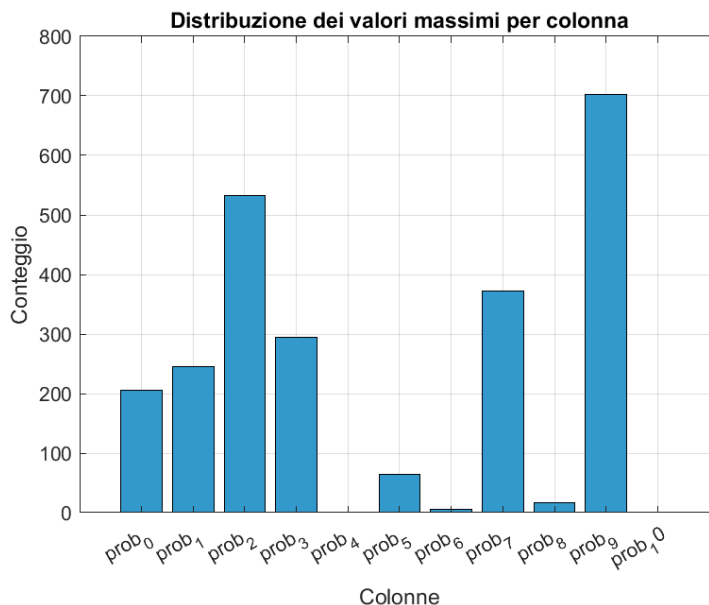


Figura 4.6: Occorrenze file submission_validation_cnn_svm.m

84,78%, che riflette una buona generalizzazione ai dati non visti durante l'addestramento. L'andamento dell'accuratezza e della funzione di costo durante il processo di training conferma che l'ottimizzazione ha proceduto in modo efficace, con una progressiva riduzione dell'errore e un miglioramento delle performance di classificazione.

Infine, l'analisi della matrice di confusione evidenzia che il modello ha effettuato un buon lavoro di classificazione.